

4. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

```

GNU nano 0.7.1 practical4a.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid[3];
    int i, status;

    // Create 3 child processes
    for (i = 0; i < 3; i++) {
        pid[i] = fork();

        if (pid[i] < 0) {
            printf("Fork failed!\n");
            exit(1);
        }

        if (pid[i] == 0) {
            // Child Process
            printf("Child %d started\n", i + 1);
            printf("PID = %d\n", getpid());
            printf("PPID = %d\n", getppid());

            sleep(i + 2); // Different execution times

            printf("Child %d completed\n", i + 1);
            exit(0);
        }
    }

    // Parent waits for all children using wait()
    printf("\nParent waiting using wait()\n\n");

    for (i = 0; i < 3; i++) {
        wait(&status);
        printf("A child process terminated.\n");
    }

    printf("\nAll child processes completed using wait().\n");

    return 0;
}

```

```

thumu_hemanth_kumar@Hen X + v
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~$ nano practical4a.c
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~$ gcc practical4a.c -o practical4a
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~$ ./practical4a
Child 1 started
Child 2 started

PID = 595
PID = 594
Parent waiting using wait()
PPID = 593
PPID = 593

Child 3 started
PID = 596
PPID = 593
Child 1 completed

A child process terminated.
Child 2 completed

A child process terminated.
Child 3 completed

A child process terminated.

All child processes completed using wait().
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~$ |

```